

Tecnología Digital VI

Forward y Backward Propagation

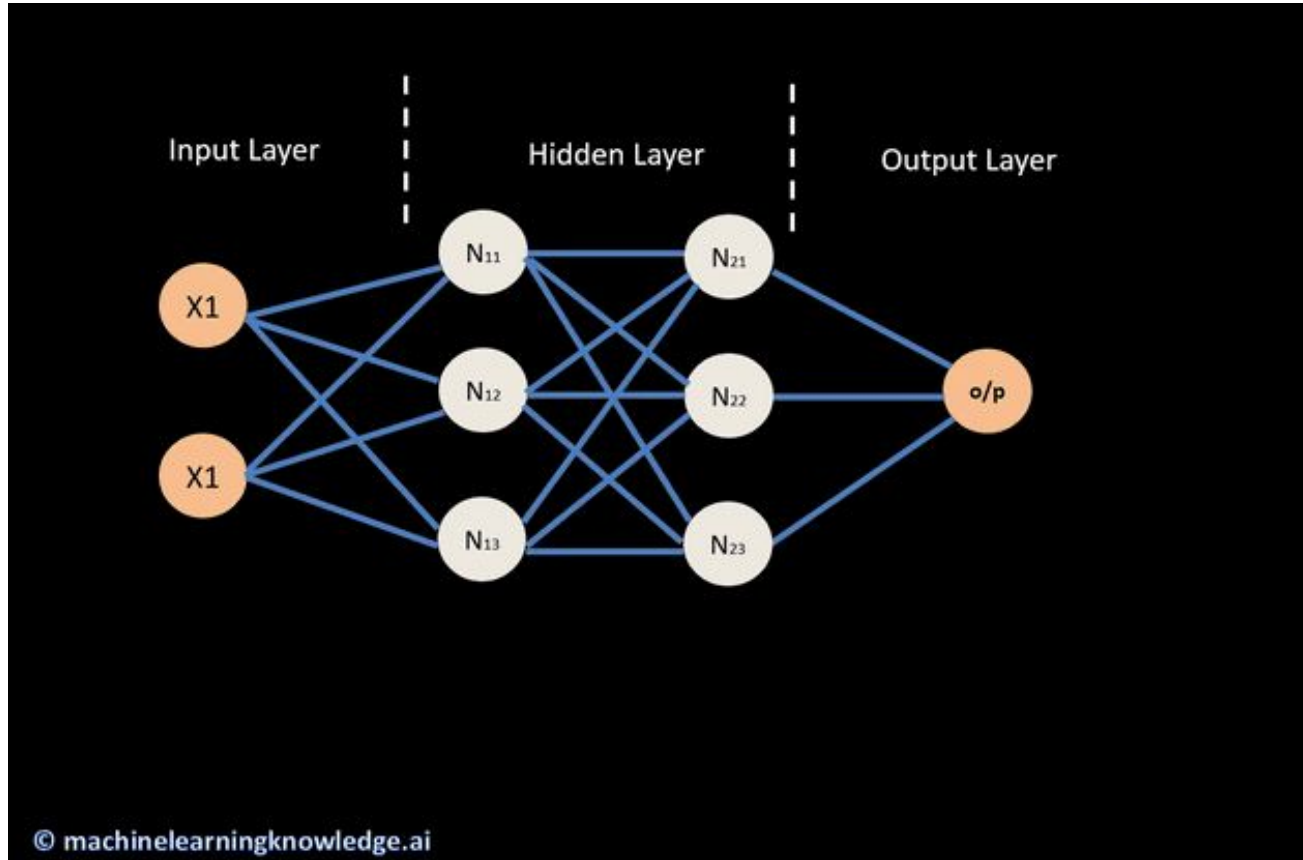
Licenciatura en Tecnología Digital - UTDT

Modos de uso

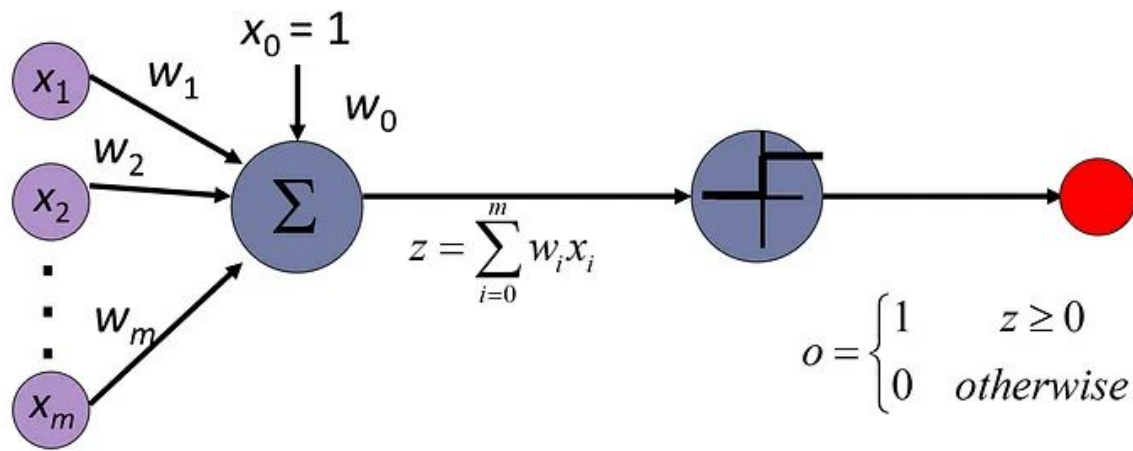
Las Redes Neuronales tienen principalmente 2 modos de uso:

- Entrenamiento: El propósito de esta etapa es optimizar los parámetros de la red (pesos y biases) para predecir correctamente el conjunto de entrenamiento.
 - Usamos tanto Forward como Backward Propagation. Con Forward sabemos qué tan mal estamos con respecto a la *ground truth*. Con Backward ajustamos los pesos en consecuencia.
- Inferencia: El propósito de esta etapa es, utilizando una red ya entrenada, realizar predicciones sobre nuevos datos.
 - Solamente pasamos los datos *hacia adelante*, utilizando Forward Propagation.

Forward y Backward Propagation



Forward Pass



$$y = \varphi \left(\sum_{j=0}^m w_j x_j \right)$$

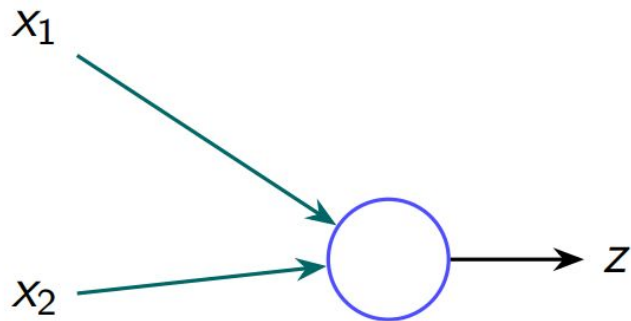
Forward Pass

Ejemplo: 2 inputs y 1 unidad.

- **Combinación lineal:**

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

- x_1, x_2 son los inputs.
- w_1, w_2 son los pesos (weights).
- b es el término bias.
- z es la combinación lineal para la unidad (previo a la función de activación).



Más outputs

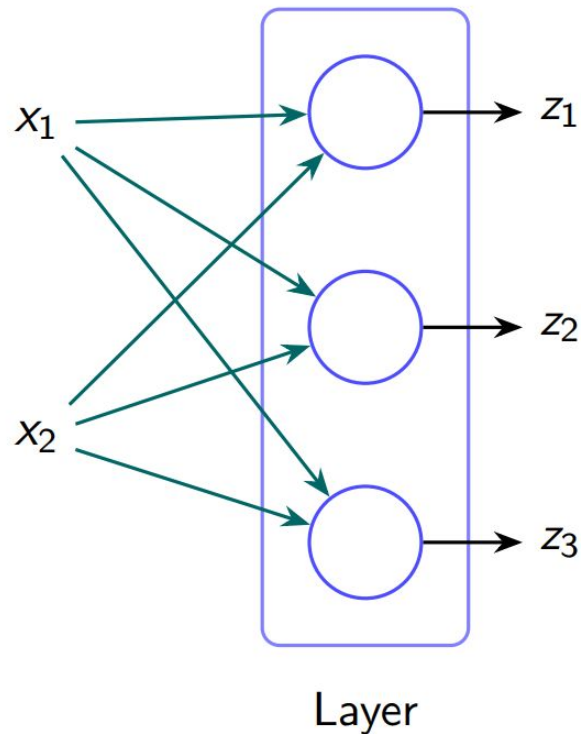
Ejemplo: 2 inputs y 3 unidades.

- **Combinación lineal por unidad:**

$$z_1 = w_{11} \cdot x_1 + w_{21} \cdot x_2 + b_1$$

$$z_2 = w_{12} \cdot x_1 + w_{22} \cdot x_2 + b_2$$

$$z_3 = w_{13} \cdot x_1 + w_{23} \cdot x_2 + b_3$$

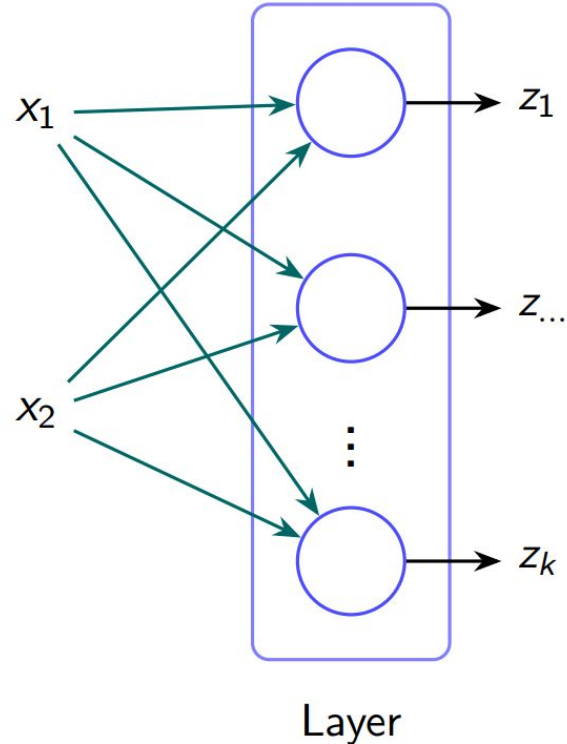


k Outputs

Ejemplo: 2 inputs y k unidades.

- Generalizamos la notación para cualquier cantidad de unidades en la capa.
- **Combinación lineal:**

$$z_k = w_{1k} \cdot x_1 + w_{2k} \cdot x_2 + b_k$$



Sumando activación

Ejemplo: 2 inputs y k unidades con Activación.

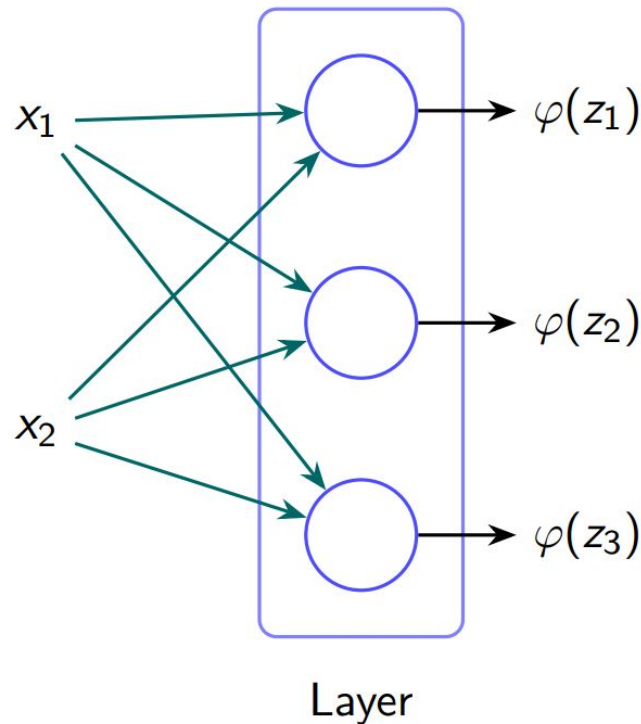
- **Combinación lineal:**

$$z_k = w_{1k} \cdot x_1 + w_{2k} \cdot x_2 + b_k$$

- **Función de activación:**

Para que la red pueda aprender patrones no lineales, pasamos z por una función φ .

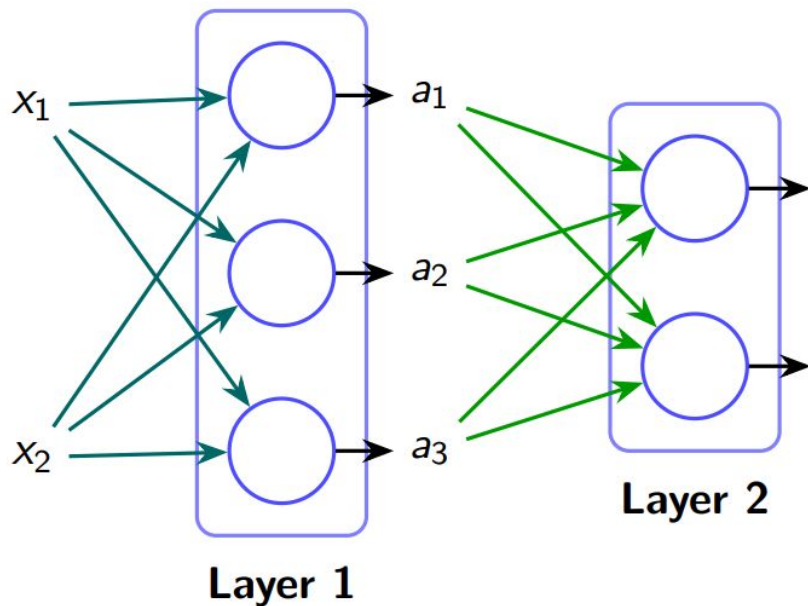
$$a_k = \varphi(z_k)$$



Uniendo capas

Uniendo todo: La salida de una capa es la entrada de la siguiente.

- **Capa 1:** Computa su salida a_k .
 $a_k = \varphi(z_k)$
- **Capa 2 (Hidden):**
Toma los a_k como sus nuevos inputs x .
Cada capa procesa secuencialmente hasta llegar a la salida final $\hat{y}$.



Vectorización

Hasta ahora parecen todas cuentas independientes con sumatorias y otras funciones.

Podemos reescribir estas cuentas para pasar a manipular matrices y vectores.

- Los pesos W de una capa, podemos pensarlos como un matriz.
- x es el vector con todos los inputs.
- b es el vector con los biases de cada neurona.

De esta manera, el *forward pass* de una capa lineal es $z = W^t x + b$

Vectorización

Si tenemos 2 inputs y 3 unidades en la capa:

- Forma extendida:

$$z_1 = w_{11}x_1 + w_{21}x_2 + b_1$$

$$z_2 = w_{12}x_1 + w_{22}x_2 + b_2$$

$$z_3 = w_{13}x_1 + w_{23}x_2 + b_3$$

- Forma vectorizada:

$$z = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Vectorización

El manejo de redes neuronales no es otra cosa que sucesivas multiplicaciones de matrices y aplicaciones de funciones sobre vectores.

Esto se puede manejar con cálculo vectorial paralelizable con varios beneficios:

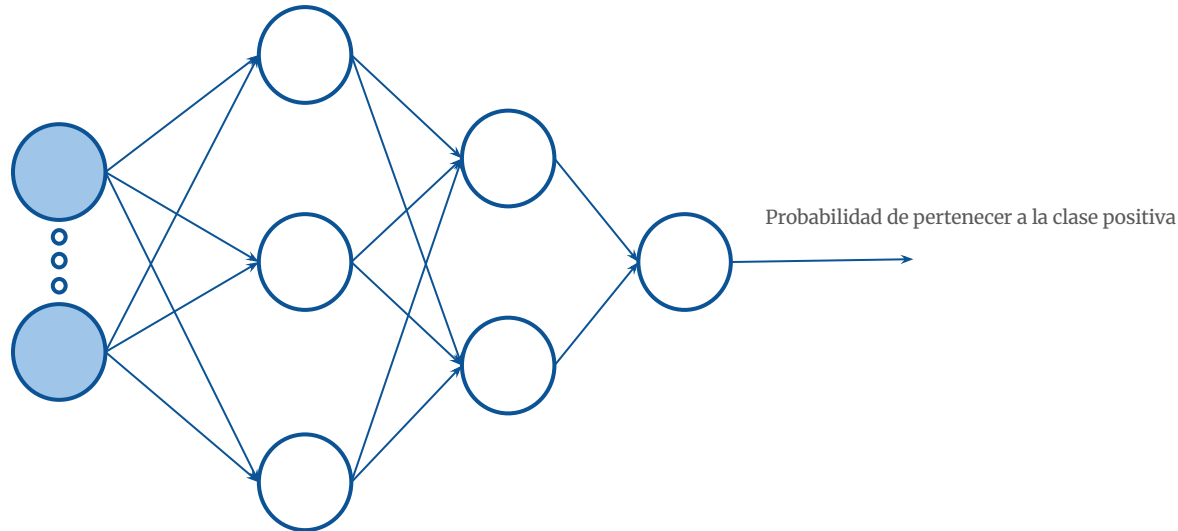
- Escalabilidad: posibilidad de manejar redes mucho más grandes.
- Claridad: Simplifica la notación.
- Performance: Aceleración de cómputo mediante GPUs y librerías optimizadas.

Existen varias librerías y frameworks que manejan este tipo de cálculos de manera muy optimizada como PyTorch, TensorFlow, Keras, etc.

Backpropagation

Ya sabemos *ir hacia adelante*, ¿cómo vamos *hacia atrás*?

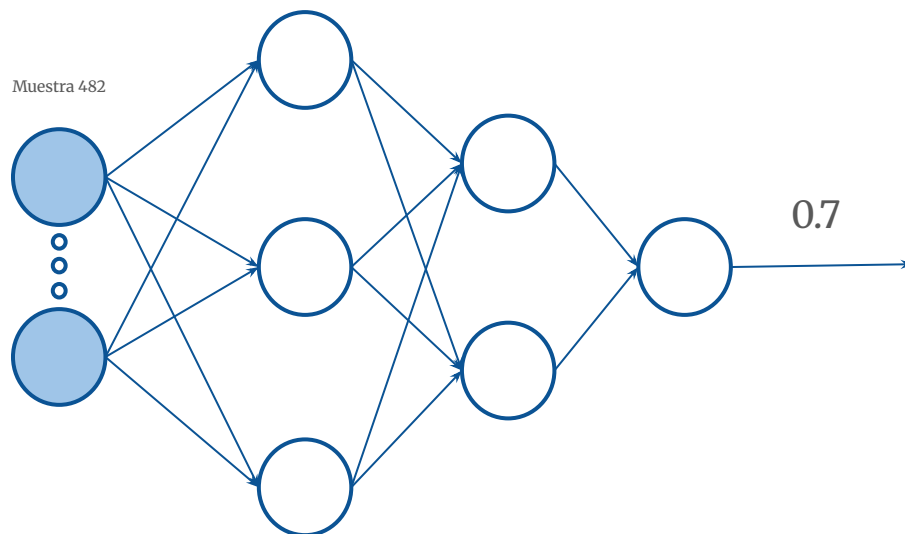
Supongamos una pequeña red de ejemplo para una clasificación binaria. Esta red transforma el input a través de dos capas ocultas hasta la capa de salida.



Backpropagation

Tomo una muestra positiva cualquiera y *la paso* por la red. Me gustaría que de 1.0, pero me dió 0.7.

La red no está tan bien como me gustaría, ¿cómo mido qué tan mal está?



Backpropagation

Al igual que usabamos en Regresión Logística, acá podemos utilizar la Negative Log Likelihood (o BCE en Redes Neuronales) para medir qué tan mal están las probabilidades arrojadas con respecto a las labels reales.

$$\mathcal{J} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Gradient Descent *a mano*

La clase LogisticRegression de sklearn ya tiene sus optimizadores implementados adentro.

¿Qué necesitamos si queremos hacer nosotros el solver?

- Las derivadas parciales de la función de costo.
- Un learning rate para hacer un paso de la actualización de los pesos.

Let's start by calculating the partial derivative of the log-likelihood function with respect to the j th weight:

$$\frac{\partial}{\partial w_j} l(\mathbf{w}) = \left(y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z)$$

Before we continue, let's also calculate the partial derivative of the sigmoid function:

$$\frac{\partial}{\partial z} \phi(z) = \frac{\partial}{\partial z} \frac{1}{1+e^{-z}} = \frac{1}{(1+e^{-z})^2} e^{-z} = \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}} \right) = \phi(z)(1-\phi(z))$$

Now, we can resubstitute $\frac{\partial}{\partial z} \phi(z) = \phi(z)(1-\phi(z))$ in our first equation to obtain the following:

$$\begin{aligned} \left(y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z) &= \left(y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \phi(z)(1-\phi(z)) \frac{\partial}{\partial w_j} z \\ &= \left(y(1-\phi(z)) - (1-y)\phi(z) \right) x_j \\ &= (y - \phi(z)) x_j \end{aligned}$$

Remember that the goal is to find the weights that maximize the log-likelihood so that we perform the update for each weight as follows:

$$w_j := w_j + \eta \sum_{i=1}^n \left(y^{(i)} - \phi(z^{(i)}) \right) x_j^{(i)}$$

Backpropagation

En una Regresión Logística las cuentas completas eran simples y podíamos derivar parcialmente la fórmula completa. En esta red, si queremos optimizar todos los W , la fórmula es bastante más compleja porque de hecho la utilización de los W no sucede al *mismo tiempo* y están *encadenados*.

Arranquemos al menos con un objetivo más conservador, y veamos cómo deberíamos cambiar los pesos de la última capa (la más cercana al output) para mejorar nuestra *loss function*.

Una opción es hacer GD solo para esos parámetros.

$$W^{[3]} = W^{[3]} - \alpha \frac{\partial \mathcal{J}}{\partial W^{[3]}}$$

Backpropagation

Para poder realizar la derivada de J respecto de los W^3 veamos por un lado todas las variables y datos que entran en juego para calcular el output de la última capa. Por otro lado, para simplificar un poco la cuestión en vez de trabajar con J (sobre todas las muestras), trabajemos sobre L (sobre una muestra).

$$z^{[3]} = W^{[3]}a^{[2]} + b^{[3]}$$

$$a^{[3]} = \sigma(z^{[3]}) = \hat{y}$$

$$\mathcal{J}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Backpropagation

Con la notación presentada, lo que queremos entonces calcular es la derivada parcial de L con respecto a los pesos de la capa de salida.

$$\frac{\partial \mathcal{L}}{\partial W^{[3]}} = - \left[y \frac{\partial}{\partial W^{[3]}} \log(a^{[3]}) + (1 - y) \frac{\partial}{\partial W^{[3]}} \log(1 - a^{[3]}) \right]$$

Backpropagation

$$\frac{\partial}{\partial z} \sigma(z) = \sigma(z)(1 - \sigma(z))$$

Usando la derivada de la sigmoidea que ya conocemos y la regla de la cadena para derivar funciones compuestas, podemos calcular la derivada que necesitamos.

$$\begin{aligned} & - \left[y \frac{1}{a^{[3]}} a^{[3]} (1 - a^{[3]}) a^{[2]T} + (1 - y) \frac{1}{1 - a^{[3]}} (-1) a^{[3]} (1 - a^{[3]}) a^{[2]T} \right] \\ & = - \left[y (1 - a^{[3]}) a^{[2]T} - (1 - y) a^{[3]} a^{[2]T} \right] \\ & = - \left[ya^{[2]T} - ya^{[3]} a^{[2]T} - a^{[3]} a^{[2]T} + ya^{[3]} a^{[2]T} \right] \\ & = - \left[ya^{[2]T} - a^{[3]} a^{[2]T} \right] = (a^{[3]} - y) a^{[2]T} \end{aligned}$$

Backpropagation

No vamos a ver la derivada en detalle, pero es interesante remarcar dos cosas:

- La derivada depende de y (dato) y de a^2 y a^3 (valores calculables en Forward Propagation).
- Una vez que tenemos la derivada de L es simple escribir la de J .

$$\frac{\partial \mathcal{J}}{\partial W^{[3]}} = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} - a^{[3]}) a^{[2]T}$$

Backpropagation

Todo este esfuerzo fue para conseguir las derivadas que nos permiten actualizar los pesos de la capa de salida, ¿y los pesos de las demás capas?

Vamos a tratar de conseguir la derivada parcial de la *loss function* con respecto a los pesos de la segunda capa oculta (la anterior a la de la salida).

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}}$$

Backpropagation

En este punto (y mucho más en lo que sigue) sería muy engorroso escribir el output de la red en función de estos pesos porque sería una composición de funciones muy grande. Vamos a aprovechar la regla de la cadena para calcular la derivadas que necesitamos en función de otras derivadas de los pasos intermedios (intermedios entre W^2 e y).

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[3]}} \frac{\partial a^{[3]}}{\partial z^{[3]}} \frac{\partial z^{[3]}}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z^{[2]}} \frac{\partial z^{[2]}}{\partial W^{[2]}}$$

Backpropagation

Algunas de estas partes ya las calculamos en un paso anterior, y otras son fáciles de caracterizar.

$$\frac{\partial \mathcal{J}}{\partial W^{[2]}} = \underbrace{\frac{\partial \mathcal{J}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}}}_{(a^{[3]} - y)} \cdot \overbrace{\frac{\partial z^{[3]}}{\partial a^{[2]}}}^{W^{[3]T}} \cdot \underbrace{\frac{\partial a^{[2]}}{\partial z^{[2]}}}_{a^{[2]}(1 - a^{[2]})} \cdot \overbrace{\frac{\partial z^{[2]}}{\partial W^{[2]}}}^{a^{[1]T}}$$

Backpropagation

Más allá de toda la derivación, finalmente podemos ver que esto vuelve a depender de valores conocidos y ya calculados:

- a^3, a^2, a^1 : Todos valores calculados al *pasar para adelante*.
- y : nuestra *ground truth*.
- W^3 : los pesos de la red.

Backpropagation

Ahora sí estamos en condiciones de calcular las derivadas parciales con respecto a todos los pesos de la red. De esta manera, luego podemos utilizar estas derivadas en un algoritmo optimizador de la *loss function* como podría ser Gradient Descent.

Sin embargo, hay un *pequeñísimo* problema. Todo este esfuerzo fue para conseguir el gradiente de la Negative Log Likelihood o Binary Cross Entropy. ¿Qué pasa cuando cambie la loss function en un problema distinto? ¿Qué se les ocurre que se puede hacer en la práctica?

Grafos Computacionales

Las librerías de Redes Neuronales están preparadas para derivar *cualquier cosa*. ¿Cómo hacen esto? No derivan expresiones grandes de manera particularizada sino que se aprovechan de las reglas de diferenciación y de la regla de la cadena para partir las fórmulas en pequeñas componentes fundamentales.

Esta solución es lo que se denomina *Diferenciación Automática*. Cada librería de Redes Neuronales tiene su propia implementación interna para poder entrenar cualquier tipo de red. En el caso de PyTorch, esto lo hace el módulo AutoGrad.

Grafos Computacionales

Lo que hace AutoGrad es partir todas las operaciones necesarias en lo que se llama un *grafo computacional*.

Estos grafos indican las operaciones fundamentales que el motor debe utilizar y el orden de dependencias para hacerlo.

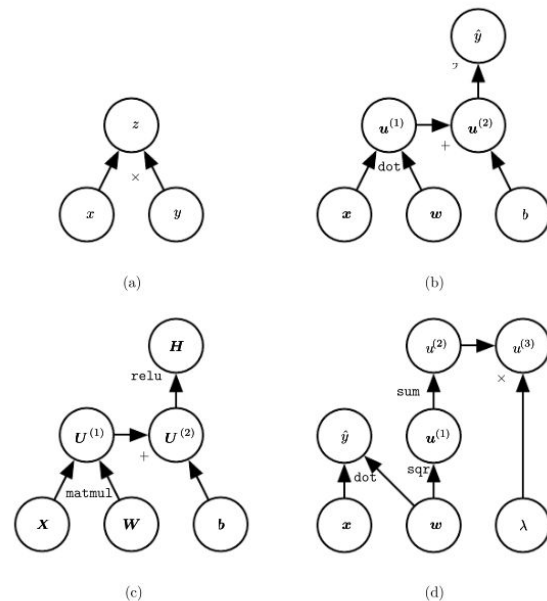


Figure 6.8: Examples of computational graphs. (a) The graph using the $\times$ operation to compute $z = xy$. (b) The graph for the logistic regression prediction $\hat{y} = \sigma(\mathbf{x}^\top \mathbf{w} + b)$. Some of the intermediate expressions do not have names in the algebraic expression but need names in the graph. We simply name the i -th such variable $u^{(i)}$. (c) The computational graph for the expression $\mathbf{H} = \max\{0, \mathbf{XW} + \mathbf{b}\}$, which computes a design matrix of rectified linear unit activations $\mathbf{H}$ given a design matrix containing a minibatch of inputs $\mathbf{X}$. (d) Examples a-c applied at most one operation to each variable, but it is possible to apply more than one operation. Here we show a computation graph that applies more than one operation to the weights $\mathbf{w}$ of a linear regression model. The weights are used to make both the prediction $\hat{y}$ and the weight decay penalty $\lambda \sum_i w_i^2$.

Grafos Computacionales

PyTorch permite acceder al Grafo Computacional de la red y visualizarlo.

```
# Define the sequential model
model = nn.Sequential(
    nn.Linear(X_train.shape[1], 1),
    nn.Sigmoid()
)

# Define the loss function and optimizer
criterion = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

from torchviz import make_dot
grafo = make_dot(loss, params=dict(model.named_parameters()))
grafo.render("grafo_breast_cancer", format="png")
```

